LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK & QUEUE

Oleh:

Nor Khalisyah NIM.2510817120005

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack & Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Nor Khalisyah
NIM : 2510817120005

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 1
B. Pembahasan 2
SOAL 2 6
A. Source Code 7
B. Output Program 8
C. Pembahasan 8
SOAL 3 11
A. Source Code 11
B. Pembahasan 12
SOAL 4 17
A. Source Code 18
B. Output Program 19
C. Pembahasan 19
TAUTAN GIT 21

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban prob1.cpp 8
Gambar 2. Screenshot Hasil Jawaban prob2.cpp 19

DAFTAR TABEL

Table 1. Source Code stack.h 1
Table 2. Source Code stack.cpp 1
Table 3. Contoh input & output soal 2 7
Table 4. Source Code prob1.cpp 7
Table 5. Source Code queue.h 11
Table 6. Source Code queue.cpp 11
Table 7. Contoh input & output soal 4 18
Table 8. Souce Code prob2.cpp 18

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1. Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Table 2. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}

bool isFull(const Stack* s) {
```

```
13      return s->top == s->data + MAX - 1;
14  }
15
16  void push(Stack* s, int value) {
17       if (isFull(s)) throw std::overflow_error("Stack is
    full");
18      *(++s->top) = value;
19  }
20
21  void pop(Stack* s) {
22       if (isEmpty(s)) throw std::underflow_error("Stack is
    empty");
23      s->top--;
24  }
25
26  int peek(const Stack* s) {
27       if (isEmpty(s)) throw std::underflow_error("Stack is
    empty");
28      return *(s->top);
29  }
```

B. Pembahasan

Pembahasan [stack.h]

Pada baris [1], digunakan `#ifndef STACK_H` dan pada baris [2] digunakan `#define STACK_H` sebagai include guard untuk mencegah file header ini di-include lebih dari satu kali dalam proses kompilasi.

Pada baris [4], digunakan `#define MAX 100` untuk mendefinisikan konstanta `MAX` bernilai 100 yang digunakan sebagai ukuran maksimum array pada struct `Stack`.

Pada baris [6] – [9], terdapat definisi struct `Stack` yang memiliki dua anggota. Anggota pertama adalah `int data[MAX]` pada baris [7] yaitu array statis berukuran `MAX` yang berfungsi sebagai tempat penyimpanan elemen-elemen stack. Anggota kedua adalah `int* top` pada baris [8] yaitu pointer bertipe integer yang selalu menunjuk ke elemen paling atas stack.

Pada baris [11], dideklarasikan fungsi `init` yang menerima pointer ke `Stack` dan berfungsi untuk menginisialisasi stack ke keadaan kosong. Pada baris [12], dideklarasikan fungsi `isEmpty` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `bool` untuk mengecek apakah stack kosong. Pada baris [13], dideklarasikan fungsi

isFull yang menerima pointer konstan ke Stack dan mengembalikan nilai bool untuk mengecek apakah stack penuh. Pada baris [14], dideklarasikan fungsi push yang menerima pointer ke Stack dan nilai int, berfungsi untuk memasukkan elemen baru ke puncak stack.

Pada baris [15], dideklarasikan fungsi pop yang menerima pointer ke Stack dan berfungsi untuk menghapus elemen paling atas stack. Pada baris [16], dideklarasikan fungsi peek yang menerima pointer konstan ke Stack dan mengembalikan nilai int, berfungsi untuk melihat elemen paling atas stack tanpa menghapusnya. Pada baris [18], digunakan #endif sebagai penutup dari include guard.

Justifikasi kebenaran: Penggunaan int* top sebagai pointer memungkinkan pointer aritmetik data - 1 sebagai sentinel kosong dan data + MAX - 1 sebagai sentinel penuh. Saat kosong top = data - 1 tidak pernah bisa diakses sebagai elemen valid. Saat ada 1 elemen top = data menunjuk elemen pertama. Saat penuh top = data + MAX - 1 menunjuk elemen terakhir. Fungsi isEmpty, isFull, dan peek menggunakan const Stack* yang membuktikan fungsi-fungsi tersebut hanya membaca data tanpa mengubah isi stack sehingga tidak ada efek samping yang tidak diinginkan.

Pembahasan [stack.cpp]

Pada baris [1], digunakan #include "stack.h" untuk menyertakan deklarasi struct Stack beserta seluruh prototipe fungsi yang akan diimplementasikan. Pada baris [2], digunakan #include <stdexcept> untuk mengaktifkan mekanisme exception handling standar C++ seperti std::overflow_error dan std::underflow_error yang digunakan untuk menangani kondisi tidak valid.

Pada baris [4] – [6], terdapat implementasi fungsi init yang menerima pointer ke Stack. Pada baris [5], pointer top diarahkan ke satu posisi sebelum data yaitu data - 1, sehingga kondisi top < data terpenuhi sebagai tanda bahwa stack dalam keadaan kosong.

Pada baris [8] – [10], terdapat implementasi fungsi isEmpty yang menerima pointer konstan ke Stack dan mengembalikan nilai bool. Pada baris [9], dilakukan

perbandingan `s->top < s->data`. Jika `top` berada di posisi lebih rendah dari `data`, fungsi mengembalikan `true` yang berarti stack kosong, dan sebaliknya mengembalikan `false`.

Pada baris [12] – [14], terdapat implementasi fungsi `isFull` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `bool`. Pada baris [13], dilakukan perbandingan apakah `top` sudah berada di posisi `data + MAX - 1` yaitu posisi terakhir array. Jika ya, berarti stack sudah penuh dan fungsi mengembalikan `true`.

Pada baris [16] – [19], terdapat implementasi fungsi `push` yang menerima pointer ke `Stack` dan nilai `int`. Pada baris [17], dilakukan pengecekan apakah stack sudah penuh menggunakan fungsi `isFull`; jika penuh maka dilempar exception `std::overflow_error` dengan pesan "Stack is full". Pada baris [18], pointer `top` diinkremen terlebih dahulu menggunakan pre-increment, kemudian nilai `value` disimpan ke lokasi yang ditunjuk `top` menggunakan dereferencing.

Pada baris [21] – [24], terdapat implementasi fungsi `pop` yang menerima pointer ke `Stack`. Pada baris [22], dilakukan pengecekan apakah stack kosong menggunakan fungsi `isEmpty`; jika kosong maka dilempar exception `std::underflow_error` dengan pesan "Stack is empty". Pada baris [23], pointer `top` didekremen satu posisi ke belakang sehingga elemen teratas dianggap telah dihapus.

Pada baris [26] – [29], terdapat implementasi fungsi `peek` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `int`. Pada baris [27], dilakukan pengecekan apakah stack kosong; jika kosong maka dilempar exception `std::underflow_error`. Pada baris [28], dikembalikan nilai yang ditunjuk oleh pointer `top` menggunakan dereferencing tanpa mengubah state stack.

Justifikasi kebenaran: Penggunaan satu pointer `top` terbukti benar untuk merepresentasikan stack karena sesuai dengan prinsip LIFO (Last In First Out). `top` selalu menunjuk elemen yang terakhir masuk dan pertama keluar. Kondisi kosong ditandai dengan `top < data` yang terpenuhi setelah `init`, dan kondisi penuh ditandai dengan `top == data + MAX - 1` yang terpenuhi ketika seluruh slot array terisi. Pre-increment pada `push` dan decrement pada `pop` menjamin konsistensi posisi `top` sehingga terbukti benar bahwa stack beroperasi sesuai prinsip LIFO.

Analisis kompleksitas: Seluruh fungsi yaitu `init`, `isEmpty`, `isFull`, `push`, `pop`, dan `peek` masing-masing memiliki kompleksitas waktu O(1) karena hanya melakukan operasi konstan berupa assignment, perbandingan, atau increment/decrement pointer tanpa perulangan. Seluruh fungsi juga memiliki kompleksitas ruang O(1) karena tidak mengalokasikan memori tambahan di luar struct `Stack` yang sudah ada.

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Table 3. Contoh input & output soal 2

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Table 4. Source Code prob1.cpp

```
#include "stack.h"
#include <iostream>
#include <string>

int main() {
    int n;
    std::cin >> n;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        std::string token;
        std::cin >> token;

        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int b = peek(&s); pop(&s);
            int a = peek(&s); pop(&s);
            int result;
            if (token == "+") result = a + b;
            else if (token == "-") result = a - b;
            else if (token == "*") result = a * b;
            else result = a / b;
```

```
24                 push(&s, result);
25             } else {
26                 push(&s, std::stoi(token));
27             }
28         }
29
30         std::cout << peek(&s) << std::endl;
31         return 0;
32     }
```

B. Output Program

Gambar 1. Screenshot Hasil Jawaban prob1.cpp

C. Pembahasan

Pada baris [1], digunakan `#include "stack.h"` untuk menyertakan deklarasi struct `Stack` beserta seluruh fungsi stack yang dibutuhkan. Pada baris [2], digunakan `#include <iostream>` untuk mengaktifkan operasi input dan output standar C++. Pada baris [3], digunakan `#include <string>` untuk mengaktifkan penggunaan tipe data `std::string` dan fungsi konversi `std::stoi`.

Pada baris [5], terdapat fungsi `main` sebagai titik masuk program. Pada baris [6] – [7], dideklarasikan variabel `n` bertipe `int` kemudian dibaca dari input standar menggunakan `std::cin`. Variabel `n` merepresentasikan jumlah token yang akan diproses.

Pada baris [9], dideklarasikan objek `Stack s`. Pada baris [10], stack `s` diinisialisasi menggunakan fungsi `init` sehingga berada dalam keadaan kosong.

Pada baris [12], dimulai perulangan sebanyak `n` iterasi untuk memproses setiap token input satu per satu. Pada baris [13] – [14], dideklarasikan variabel `token` bertipe `std::string` kemudian dibaca dari input standar.

Pada baris [16], dilakukan pengecekan apakah token merupakan salah satu operator aritmetika yaitu `+`, `-`, `*`, atau `/`. Jika ya, pada baris [17], diambil nilai teratas stack menggunakan `peek` dan disimpan ke variabel `b`, kemudian elemen tersebut dihapus menggunakan `pop`. Pada baris [18], diambil kembali nilai teratas stack yang sekarang menjadi operan kedua dan disimpan ke variabel `a`, kemudian elemen tersebut juga dihapus. Perlu diperhatikan bahwa `b` diambil lebih dulu karena merupakan operan kanan, sedangkan `a` adalah operan kiri sesuai urutan notasi postfix. Pada baris [19], dideklarasikan variabel `result` untuk menyimpan hasil operasi. Pada baris [20] – [23], dilakukan operasi aritmetika sesuai dengan token operator yang dibaca, kemudian hasilnya disimpan ke `result`. Pada baris [24], hasil operasi di-push kembali ke stack.

Pada baris [25] – [27], jika token bukan operator maka token tersebut dianggap sebagai bilangan bulat. Pada baris [26], token dikonversi dari `std::string` ke `int` menggunakan fungsi `std::stoi`, kemudian di-push ke stack.

Pada baris [30], setelah seluruh token diproses, nilai teratas stack yang merupakan hasil akhir ekspresi postfix dicetak menggunakan `peek` diikuti newline dari `std::endl`. Pada baris [31], dikembalikan nilai 0 sebagai tanda bahwa program berjalan dengan sukses.

Justifikasi kebenaran: Algoritma ini mengimplementasikan evaluasi ekspresi postfix (Reverse Polish Notation) menggunakan stack. Prinsip LIFO pada stack terbukti tepat digunakan karena operan yang terakhir dimasukkan adalah yang pertama digunakan dalam operasi. Dengan mengambil `b` sebelum `a`, urutan operan tetap terjaga sesuai ekspresi aslinya sehingga operasi tidak komutatif seperti pengurangan dan pembagian menghasilkan nilai yang benar. Setelah seluruh token diproses, tepat satu elemen tersisa di stack yang merupakan hasil akhir ekspresi.

Analisis kompleksitas: Perulangan utama berjalan sebanyak `n` iterasi dan setiap iterasi hanya melakukan operasi push, pop, atau peek yang masing-masing O(1), sehingga total kompleksitas waktu adalah O(n). Kompleksitas ruang adalah O(n) karena dalam kasus terburuk seluruh token berupa operan sehingga stack menyimpan maksimal `n` elemen sekaligus.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 5. Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

Table 6. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data - 1;
}

bool isEmpty(const Queue* q) {
```

```
10      return q->rear < q->front;
11  }
12
13  bool isFull(const Queue* q) {
14      return (int)(q->rear - q->data) >= MAX - 1;
15  }
16
17  void enqueue(Queue* q, int value) {
18       if (isFull(q)) throw std::overflow_error("Queue is
    full");
19      *(++q->rear) = value;
20  }
21
22  void dequeue(Queue* q) {
23       if (isEmpty(q)) throw std::underflow_error("Queue is
    empty");
24      int count = (int)(q->rear - q->front);
25      for (int i = 0; i < count; i++) {
26           q->data[i] = q->data[i + (q->front - q->data) +
    1];
27      }
28      q->front = q->data;
29      q->rear = q->data + count - 1;
30  }
31
32  int front(const Queue* q) {
33       if (isEmpty(q)) throw std::underflow_error("Queue is
    empty");
34      return *(q->front);
35  }
36
37  int back(const Queue* q) {
38       if (isEmpty(q)) throw std::underflow_error("Queue is
39  empty");
        return *(q->rear);
40  }
```

B. Pembahasan

Pembahasan [queue.h]

Pada baris [1], digunakan `#ifndef QUEUE_H` dan pada baris [2] digunakan `#define QUEUE_H` sebagai include guard untuk mencegah file header ini di-include lebih dari satu kali dalam proses kompilasi.

Pada baris [4], digunakan `#define MAX 100` untuk mendefinisikan konstanta `MAX` bernilai 100 yang digunakan sebagai ukuran maksimum array pada struct `Queue`.

Pada baris [6] – [10], terdapat definisi struct `Queue` yang memiliki tiga anggota. Anggota pertama adalah `int data[MAX]` pada baris [7] yaitu array statis berukuran `MAX` yang berfungsi sebagai tempat penyimpanan elemen-elemen queue. Anggota kedua adalah `int* front` pada baris [8] yaitu pointer yang selalu menunjuk ke elemen paling depan queue. Anggota ketiga adalah `int* rear` pada baris [9] yaitu pointer yang selalu menunjuk ke elemen paling belakang queue.

Pada baris [12], dideklarasikan fungsi `init` yang menerima pointer ke `Queue` dan berfungsi untuk menginisialisasi queue ke keadaan kosong. Pada baris [13], dideklarasikan fungsi `isEmpty` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `bool` untuk mengecek apakah queue kosong. Pada baris [14], dideklarasikan fungsi `isFull` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `bool` untuk mengecek apakah queue penuh. Pada baris [15], dideklarasikan fungsi `enqueue` yang menerima pointer ke `Queue` dan nilai `int`, berfungsi untuk memasukkan elemen baru ke belakang queue.

Pada baris [16], dideklarasikan fungsi `dequeue` yang menerima pointer ke `Queue` dan berfungsi untuk menghapus elemen paling depan queue. Pada baris [17], dideklarasikan fungsi `front` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int` paling depan queue. Pada baris [18], dideklarasikan fungsi `back` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int` paling belakang queue. Pada baris [20], digunakan `#endif` sebagai penutup dari include guard.

Justifikasi kebenaran: Penggunaan dua pointer `front` dan `rear` terbukti benar untuk merepresentasikan queue karena sesuai dengan prinsip FIFO (First In First Out). `front` selalu menunjuk elemen yang pertama masuk dan pertama keluar, sedangkan `rear` selalu menunjuk elemen yang terakhir masuk. Kondisi kosong ditandai dengan `rear < front` yang tidak mungkin terjadi dalam kondisi normal sehingga menjadi sentinel yang aman. Array statis berukuran 100 terbukti cukup sesuai batasan soal $n \leq 100$.

Pembahasan [queue.cpp]

Pada baris [1], digunakan `#include "queue.h"` untuk menyertakan deklarasi struct `Queue` beserta seluruh prototipe fungsi yang akan diimplementasikan. Pada baris [2], digunakan `#include <stdexcept>` untuk mengaktifkan mekanisme exception handling standar C++ seperti `std::overflow_error` dan `std::underflow_error` yang digunakan untuk menangani kondisi tidak valid.

Pada baris [4] – [7], terdapat implementasi fungsi `init` yang menerima pointer ke `Queue`. Pada baris [5], pointer `front` diarahkan ke elemen pertama array `data` (indeks 0). Pada baris [6], pointer `rear` diarahkan ke satu posisi sebelum `data` yaitu `data - 1`, sehingga kondisi `rear < front` terpenuhi sebagai tanda bahwa queue dalam keadaan kosong.

Pada baris [9] – [11], terdapat implementasi fungsi `isEmpty` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `bool`. Pada baris [10], dilakukan perbandingan `q->rear < q->front`. Jika `rear` berada di posisi lebih rendah dari `front`, fungsi mengembalikan `true` yang berarti queue kosong, dan sebaliknya mengembalikan `false`.

Pada baris [13] – [15], terdapat implementasi fungsi `isFull` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `bool`. Pada baris [14], dihitung selisih antara `rear` dan `data` menggunakan casting `(int)` untuk memastikan aritmetika pointer yang aman, kemudian dibandingkan dengan `MAX - 1`. Jika selisihnya sudah mencapai atau melebihi `MAX - 1`, berarti `rear` sudah berada di posisi terakhir array dan queue dinyatakan penuh.

Pada baris [17] – [20], terdapat implementasi fungsi `enqueue` yang menerima pointer ke `Queue` dan nilai `int`. Pada baris [18], dilakukan pengecekan apakah queue sudah penuh menggunakan fungsi `isFull`; jika penuh maka dilempar exception `std::overflow_error` dengan pesan "Queue is full". Pada baris [19], pointer `rear` diinkremen terlebih dahulu menggunakan pre-increment, kemudian nilai `value` disimpan ke lokasi yang ditunjuk `rear` menggunakan dereferencing.

Pada baris [22] – [30], terdapat implementasi fungsi `dequeue` yang menerima pointer ke `Queue`. Pada baris [23], dilakukan pengecekan apakah queue kosong

menggunakan fungsi `isEmpty`; jika kosong maka dilempar exception `std::underflow_error` dengan pesan "Queue is empty". Pada baris [24], dihitung jumlah elemen yang tersisa setelah dequeue dengan rumus `count = rear - front`. Pada baris [25] – [27], seluruh elemen yang tersisa digeser ke kiri menuju indeks 0 array menggunakan perulangan sebanyak `count` iterasi. Pada baris [28], pointer `front` direset ke `data` dan pada baris [29], pointer `rear` disesuaikan ke posisi elemen terakhir yang baru yaitu `data + count - 1`.

Pada baris [32] – [35], terdapat implementasi fungsi `front` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int`. Pada baris [33], dilakukan pengecekan apakah queue kosong; jika kosong maka dilempar exception `std::underflow_error`. Pada baris [34], dikembalikan nilai yang ditunjuk oleh pointer `front` menggunakan dereferencing tanpa mengubah state queue.

Pada baris [37] – [40], terdapat implementasi fungsi `back` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int`. Pada baris [38], dilakukan pengecekan apakah queue kosong; jika kosong maka dilempar exception `std::underflow_error`. Pada baris [39], dikembalikan nilai yang ditunjuk oleh pointer `rear` menggunakan dereferencing tanpa mengubah state queue.

Justifikasi kebenaran: Penggunaan dua pointer `front` dan `rear` terbukti benar untuk merepresentasikan queue karena sesuai dengan prinsip FIFO (First In First Out). `front` selalu menunjuk elemen yang pertama masuk dan pertama keluar, sedangkan `rear` selalu menunjuk elemen yang terakhir masuk. Kondisi kosong ditandai dengan `rear < front` yang terpenuhi setelah `init` maupun setelah seluruh elemen di-dequeue. Teknik pergeseran elemen ke kiri pada fungsi `dequeue` memastikan `rear` tidak pernah melampaui batas array meskipun operasi enqueue dan dequeue dilakukan secara bergantian berkali-kali, sehingga terbukti benar bahwa queue dapat digunakan kembali setelah dikosongkan.

Analisis kompleksitas: Fungsi `init`, `isEmpty`, `isFull`, `enqueue`, `front`, dan `back` masing-masing memiliki kompleksitas waktu O(1) karena hanya melakukan operasi konstan berupa assignment atau perbandingan pointer tanpa perulangan. Fungsi `dequeue` memiliki kompleksitas waktu O(n) karena terdapat perulangan sebanyak

jumlah elemen yang tersisa dalam queue untuk menggeser seluruh elemen ke kiri. Seluruh fungsi memiliki kompleksitas ruang O(1) karena tidak mengalokasikan memori tambahan di luar struct `Queue` yang sudah ada.

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n

A1 A2 ... An

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Table 7. Contoh input & output soal 4

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 8. Souce Code prob2.cpp

```
#include "queue.h"
#include <iostream>

int main() {
    int n, k;
    std::cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) std::cin >> arr[i];

    bool first = true;

    for (int i = 0; i <= n - k; i++) {
        Queue q;
        init(&q);
       for (int j = i; j < i + k; j++) enqueue(&q, arr[j]);

        int sum = 0;
        while (!isEmpty(&q)) {
            sum += front(&q);
            dequeue(&q);
        }

        if (!first) std::cout << " ";
        std::cout << sum;
        first = false;
    }

    std::cout << std::endl;
    return 0;
}
```

B. Output Program

Gambar 2. Screenshot Hasil Jawaban prob2.cpp

C. Pembahasan

Pada baris [1], digunakan `#include "queue.h"` untuk menyertakan deklarasi struct `Queue` beserta seluruh fungsi queue yang dibutuhkan. Pada baris [2], digunakan `#include <iostream>` untuk mengaktifkan operasi input dan output standar C++.

Pada baris [4], terdapat fungsi `main` sebagai titik masuk program. Pada baris [5] – [6], dideklarasikan dua variabel `n` dan `k` bertipe `int`, kemudian dibaca dari input standar menggunakan `std::cin`. Variabel `n` merepresentasikan jumlah elemen array dan `k` merepresentasikan ukuran jendela (window) penjumlahan.

Pada baris [8] – [9], dideklarasikan array statis `arr` berukuran 100 untuk menyimpan elemen-elemen input. Kemudian dilakukan perulangan sebanyak `n` iterasi untuk membaca setiap elemen dari input standar dan menyimpannya ke dalam `arr`.

Pada baris [11], dideklarasikan variabel `bool first` dengan nilai awal `true` yang digunakan sebagai penanda untuk mengontrol pemisah spasi antar output agar tidak terdapat spasi di awal baris.

Pada baris [13], dimulai perulangan utama sebanyak `n - k + 1` iterasi yang merepresentasikan seluruh posisi jendela yang mungkin pada array. Pada baris [14],

dideklarasikan objek `Queue q` secara lokal di dalam perulangan. Pada baris [15], queue `q` diinisialisasi menggunakan fungsi `init`. Pada baris [16], dilakukan perulangan sebanyak `k` iterasi untuk memasukkan elemen-elemen pada jendela ke-`i` yaitu dari indeks `i` hingga `i + k - 1` ke dalam queue menggunakan fungsi `enqueue`.

Pada baris [18], dideklarasikan variabel `sum` bertipe `int` dengan nilai awal 0 untuk menampung hasil penjumlahan elemen dalam jendela. Pada baris [19] – [22], dilakukan perulangan selama queue tidak kosong. Setiap iterasi mengambil nilai elemen paling depan queue menggunakan fungsi `front`, menambahkannya ke `sum` pada baris [20], kemudian menghapus elemen tersebut menggunakan fungsi `dequeue` pada baris [21].

Pada baris [24], dilakukan pengecekan variabel `first`; jika bukan iterasi pertama maka dicetak spasi sebagai pemisah antar hasil. Pada baris [25], dicetak nilai `sum` hasil penjumlahan jendela ke-`i`. Pada baris [26], variabel `first` diset menjadi `false` setelah iterasi pertama selesai.

Pada baris [29], dicetak karakter newline menggunakan `std::endl` untuk mengakhiri baris output. Pada baris [30], dikembalikan nilai 0 sebagai tanda bahwa program berjalan dengan sukses.

Justifikasi kebenaran: Algoritma ini menggunakan pendekatan sliding window dengan memanfaatkan struktur data queue untuk menjumlahkan setiap subarray berukuran `k`. Dengan menggeser jendela satu posisi ke kanan pada setiap iterasi luar, seluruh kombinasi subarray berukuran `k` yang mungkin pada array `arr` pasti akan diproses tepat satu kali. Penggunaan queue memastikan elemen diproses sesuai urutan masuk (FIFO), sehingga penjumlahan terjadi secara berurutan dari kiri ke kanan sesuai dengan posisi jendela.

Analisis kompleksitas: Perulangan luar berjalan sebanyak `n - k + 1` iterasi dan perulangan dalam untuk pengisian queue berjalan sebanyak `k` iterasi, sehingga total kompleksitas waktu adalah $O((n - k + 1) * k)$ yang dalam kasus terburuk yaitu $k = n/2$ menjadi $O(n^2)$. Kompleksitas ruang adalah $O(k)$ karena setiap iterasi hanya menyimpan maksimal `k` elemen di dalam queue.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-norkhalisyah162007-crypto.git